

“深入多线程内部机理”之三



并发编程三特性（上）

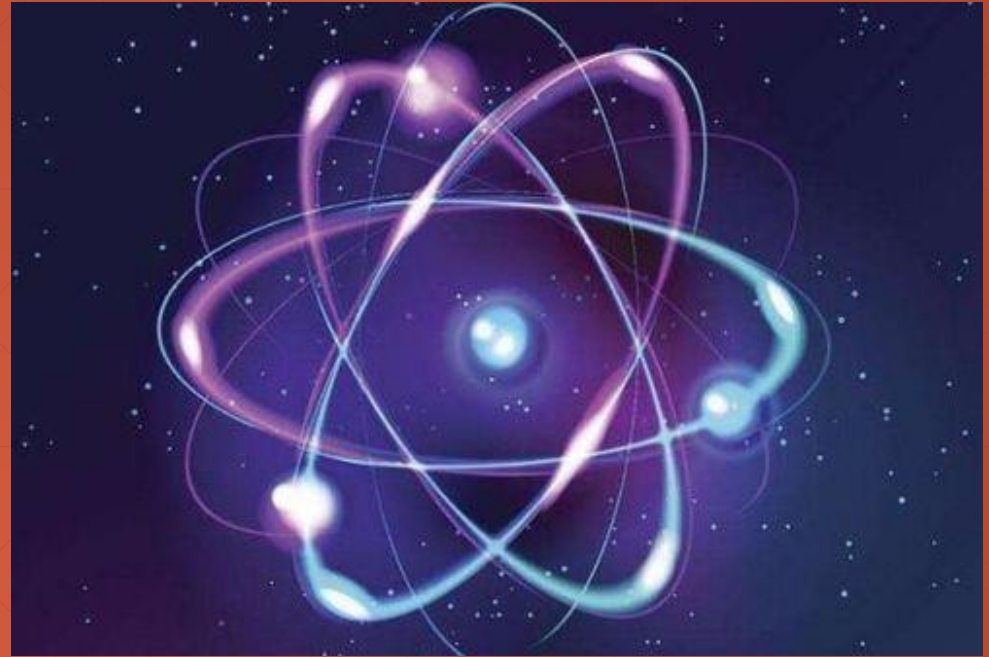
北京理工大学计算机学院
金旭亮

多线程开发要关注的三个特性

原子性

可见性

有序性



原子性

原子性操作的特点



访问（读写）某个共享变量的操作从其执行线程以外的任何线程来看，该操作要不已经执行结束，要不还未发生，其他线程不会“看到”该操作执行了部分的中间效果。

要不没开始，要不已结束，绝不能“干了一半”

Java内存模型(JMM)对操作原子性的保证



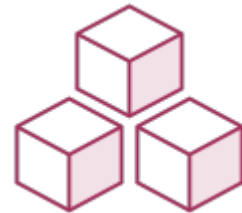
对变量简单的读取与赋值操作是原子性的，将一个变量赋给另外一个变量的操作不是原子性的。



多个原子性的操作在一起，不是原子性操作。

操作	是否原子操作
<code>x=10</code> ；赋值操作	是
<code>y=x</code> ；赋值操作	否
<code>y++</code> ；自增操作	否
<code>z=z+1</code> ；加一操作	否

Java中现有的原子操作特性及相关组件

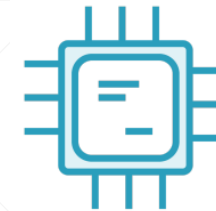


Java中，`long`和`double`类型之外的任何类型的变量的写操作都是原子的，如果给`long`和`double`类型的变量加上`volatile`关键字，则它们的写操作，也是原子的。



JDK提供了一组“原子性组件”，集中放在`atomic`包中，后面课程会有涉及和介绍。

Java代码中实现原子性的技术手段



使用“软件锁”，如同步方法或JDK中提供的锁组件。



使用处理器所提供的CAS（Compare-and-Swap）指令，俗称“硬件锁”，诸如AtomicInteger之类组件，就利用了硬件锁。



可见性

可见性



在多线程环境下，一个线程对某个共享变量进行更新之后，后继访问该变量的线程如果能读取到这个更新后的结果，我们就称这个线程对共享变量的更新对其他线程“**可见**”。

可见性就是指一个线程对某共享变量的更新的结果，对于读取这个共享变量的线程而言是否可见的问题。

引发可见性问题的原因

编译器优化（比如指令重排序）

硬件优化（CPU中的寄存器，高速缓存）

线程启动、停止时的可见性

Java语言规范（JLS, Java Language Specification）保证：

- ▶ 父线程在启动子线程之前对共享变量的更新，对于子线程来说是可见的。
- ▶ 一个线程在终止后，它对共享变量的更新，对于调用该线程的`join`方法的线程来说是可见的。

可见性示例

```
ThreadVisibility.java x
1  ▶ public class ThreadVisibility {
    3 usages
2      static int data = 0;
3  ▶  public static void main(String[] args) throws InterruptedException {
4      var thread = new Thread(() -> {
5          try {
6              Thread.sleep( millis: 50);
7              System.out.println(data);
8          } catch (InterruptedException e) {
9              throw new RuntimeException(e);
10         }
11     });
12     //这里对data的修改, 子线程是可见的
13     data = 1;
14     thread.start();
15     Thread.sleep( millis: 50);
16     //这里对data的修改, 子线程可能可以看得见,
17     //也可能看不见, 造成结果的不确定性
18     data = 2;
19 }
20 }
```

Java保障可见性的一个手段



为了保证内存可见性，Java编译器在生成指令序列的适当位置会插入**内存屏障指令**来禁止特定类型的处理器重排序。`volatile`就是非常典型的例子。